

Admin Panel — Reusable Components

1. DataTable

```
// components/ui/data-table.tsx
"use client";

import {
  ColumnDef,
  SortingState,
  flexRender,
  getCoreRowModel,
  useReactTable,
} from "@tanstack/react-table";
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "@components/ui/table";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Skeleton } from "@components/ui/skeleton";
import { ArrowUpDown, ArrowUp, ArrowDown } from "lucide-react";
import { useState } from "react";

interface PaginationState {
  page: number;
  pageSize: number;
  totalPages: number;
}

interface DataTableProps<TData, TValue> {
  columns: ColumnDef<TData, TValue>[];
  data: TData[];
  isLoading?: boolean;
  pagination?: PaginationState;
  onPaginationChange?: (page: number) => void;
  sorting?: SortingState;
  onSortingChange?: (sorting: SortingState) => void;
  searchPlaceholder?: string;
```

```

    onSearch?: (query: string) => void;
  }

export function DataTable<TData, TValue>({
  columns,
  data,
  isLoading = false,
  pagination,
  onPaginationChange,
  sorting = [],
  onSortingChange,
  searchPlaceholder = "Search...",
  onSearch,
}: DataTableProps<TData, TValue>) {
  const [searchValue, setSearchValue] = useState("");

  const table = useReactTable({
    data,
    columns,
    getCoreRowModel: getCoreRowModel(),
    manualSorting: true,
    manualPagination: true,
    state: { sorting },
    onSortingChange: (updater) => {
      const newSorting = typeof updater === "function" ?
updater(sorting) : updater;
      onSortingChange?.(newSorting);
    },
  });

  const handleSearch = (value: string) => {
    setSearchValue(value);
    onSearch?.(value);
  };

  const renderPageNumbers = () => {
    if (!pagination) return null;
    const { page, totalPages } = pagination;
    const pages: number[] = [];
    const start = Math.max(1, page - 2);
    const end = Math.min(totalPages, page + 2);
    for (let i = start; i <= end; i++) pages.push(i);

    return pages.map((p) => (
      <Button
        key={p}
        variant={p === page ? "default" : "outline"}
        size="sm"
        onClick={() => onPaginationChange?.(p)}
      >
        {p}
      </Button>
    ));
  };
};

```

```

return (
  <div className="space-y-4">
    {onSearch && (
      <Input
        placeholder={searchPlaceholder}
        value={searchValue}
        onChange={(e) => handleSearch(e.target.value)}
        className="max-w-sm"
      />
    )}

    <div className="rounded-md border">
      <Table>
        <TableHeader>
          {table.getHeaderGroups().map((headerGroup) => (
            <TableRow key={headerGroup.id}>
              {headerGroup.headers.map((header) => {
                const isSortable = header.column.getCanSort();
                const sorted = header.column.getIsSorted();
                return (
                  <TableHead
                    key={header.id}
                    className={isSortable ? "cursor-pointer
select-none" : ""}
                    onClick={header.column.getToggleSortingHandler()}
                  >
                    <div className="flex items-center gap-1">
                      {header.isPlaceholder
                        ? null
                        :
flexRender(header.column.columnDef.header, header.getContext())}
                      {isSortable && (
                        <>
                          {sorted === "asc" && <ArrowUp
className="h-4 w-4" />}
                          {sorted === "desc" && <ArrowDown
className="h-4 w-4" />}
                          {!sorted && <ArrowUpDown
className="h-4 w-4 text-muted-foreground" />}
                        </>
                      )}
                    </div>
                  </TableHead>
                );
              )}
            </TableRow>
          )}
        </TableHeader>
        <TableBody>
          {isLoading ? (
            Array.from({ length: pagination?.pageSize ??
10 }).map((_, i) => (

```

```

        <TableRow key={`skeleton-${i}`}>
          {columns.map((_, j) => (
            <TableCell key={j}>
              <Skeleton className="h-5 w-full" />
            </TableCell>
          ))}
        </TableRow>
      ))
    ) : table.getRowModel().rows.length ? (
      table.getRowModel().rows.map((row) => (
        <TableRow key={row.id}>
          {row.getVisibleCells().map((cell) => (
            <TableCell key={cell.id}>
              {flexRender(cell.column.columnDef.cell,
cell.getContext())}
            </TableCell>
          ))}
        </TableRow>
      ))
    ) : (
      <TableRow>
        <TableCell colSpan={columns.length}
className="h-24 text-center text-muted-foreground">
          No results found.
        </TableCell>
      </TableRow>
    )}
  </TableBody>
</Table>
</div>

{pagination && pagination.totalPages > 1 && (
  <div className="flex items-center justify-between">
    <p className="text-sm text-muted-foreground">
      Page {pagination.page} of {pagination.totalPages}
    </p>
    <div className="flex items-center gap-1">
      <Button
        variant="outline"
        size="sm"
        disabled={pagination.page <= 1}
        onClick={() => onPaginationChange?.(pagination.page
- 1)}
      >
        Previous
      </Button>
      {renderPageNumbers()}
      <Button
        variant="outline"
        size="sm"
        disabled={pagination.page >= pagination.totalPages}
        onClick={() => onPaginationChange?.(pagination.page
+ 1)}
      >

```

```

        Next
      </Button>
    </div>
  </div>
) }
</div>
);
}

```

2. KPICard

```

// components/dashboard/kpi-card.tsx
"use client";

import { Card, CardContent } from "@components/ui/card";
import { Skeleton } from "@components/ui/skeleton";
import { LucideIcon, TrendingUp, TrendingDown } from "lucide-react";
import { cn } from "@lib/utils";

interface KPICardProps {
  title: string;
  value: string | number;
  icon: LucideIcon;
  trend?: { value: number; isPositive: boolean } | null;
  isLoading?: boolean;
}

export function KPICard({ title, value, icon: Icon, trend,
  isLoading = false }: KPICardProps) {
  if (isLoading) {
    return (
      <Card>
        <CardContent className="p-6">
          <div className="flex items-center justify-between">
            <Skeleton className="h-5 w-5 rounded" />
          </div>
          <Skeleton className="mt-3 h-8 w-24" />
          <Skeleton className="mt-2 h-4 w-32" />
        </CardContent>
      </Card>
    );
  }

  return (
    <Card>
      <CardContent className="p-6">
        <div className="flex items-center justify-between">
          <Icon className="h-5 w-5 text-muted-foreground" />
          {trend && (
            <div

```

```

        className={cn(
          "flex items-center gap-1 text-xs font-medium",
          trend.isPositive ? "text-green-600" : "text-
red-600"
        )}
      >
        {trend.isPositive ? (
          <TrendingUp className="h-3 w-3" />
        ) : (
          <TrendingDown className="h-3 w-3" />
        )}
        {Math.abs(trend.value)}%
      </div>
    )}
  </div>
  <div className="mt-3">
    <p className="text-2xl font-bold tracking-tight">{value}</p>
</p>
    <p className="text-sm text-muted-foreground">{title}</p>
  </div>
</CardContent>
</Card>
);
}

```

3. StatusBadge

```

// components/ui/status-badge.tsx
import { Badge } from "@components/ui/badge";
import { cn } from "@lib/utils";

interface StatusBadgeProps {
  status: string;
  variant?: "default" | "outline";
}

const statusColorMap: Record<string, string> = {
  pending: "bg-yellow-100 text-yellow-800 border-yellow-200",
  scheduled: "bg-yellow-100 text-yellow-800 border-yellow-200",
  active: "bg-green-100 text-green-800 border-green-200",
  online: "bg-green-100 text-green-800 border-green-200",
  completed: "bg-blue-100 text-blue-800 border-blue-200",
  approved: "bg-blue-100 text-blue-800 border-blue-200",
  cancelled: "bg-red-100 text-red-800 border-red-200",
  rejected: "bg-red-100 text-red-800 border-red-200",
  suspended: "bg-red-100 text-red-800 border-red-200",
  under_review: "bg-orange-100 text-orange-800 border-orange-200",
  processing: "bg-orange-100 text-orange-800 border-orange-200",
};

function formatStatus(status: string): string {

```

```

    return status.replace(/_/g, " ").replace(/\b\w/g, (c) =>
c.toUpperCase());
}

export function StatusBadge({ status, variant = "outline" }:
StatusBadgeProps) {
  const colorClass = statusColorMap[status.toLowerCase()] ?? "bg-
gray-100 text-gray-800 border-gray-200";

  return (
    <Badge variant={variant} className={cn("font-medium",
colorClass)}>
      {formatStatus(status)}
    </Badge>
  );
}

```

4. GuardVerificationCard

```

// components/guards/guard-verification-card.tsx
"use client";

import { useState } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/
components/ui/card";
import { Button } from "@components/ui/button";
import { Textarea } from "@components/ui/textarea";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
  DialogTrigger,
} from "@components/ui/dialog";
import { StatusBadge } from "@components/ui/status-badge";
import { Check, X } from "lucide-react";

interface VerificationDocument {
  id: string;
  type: string;
  file_url: string;
  status: string;
}

interface GuardVerificationCardProps {
  document: VerificationDocument;
  onApprove: (docId: string) => void;
  onReject: (docId: string, reason: string) => void;
}

```

```

export function GuardVerificationCard({ document, onApprove,
onReject }: GuardVerificationCardProps) {
  const [rejectionReason, setRejectionReason] = useState("");
  const [dialogOpen, setDialogOpen] = useState(false);

  const handleReject = () => {
    if (!rejectionReason.trim()) return;
    onReject(document.id, rejectionReason);
    setRejectionReason("");
    setDialogOpen(false);
  };

  return (
    <Card>
      <CardHeader className="pb-3">
        <div className="flex items-center justify-between">
          <CardTitle className="text-sm font-
medium">{document.type}</CardTitle>
          <StatusBadge status={document.status} />
        </div>
      </CardHeader>
      <CardContent className="space-y-4">
        <a href={document.file_url} target="_blank" rel="noopener
noreferrer">
          <img
            src={document.file_url}
            alt={document.type}
            className="h-40 w-full rounded-md border object-cover
hover:opacity-80 transition-opacity"
          />
        </a>

        {document.status === "pending" && (
          <div className="flex gap-2">
            <Button
              size="sm"
              className="flex-1 bg-green-600 hover:bg-green-700
text-white"
              onClick={() => onApprove(document.id)}
            >
              <Check className="mr-1 h-4 w-4" />
              Approve
            </Button>

            <Dialog open={dialogOpen} onOpenChange={setDialogOpen}
>
              <DialogTrigger asChild>
                <Button size="sm" variant="destructive"
className="flex-1">
                  <X className="mr-1 h-4 w-4" />
                  Reject
                </Button>
              </DialogTrigger>
            </div>
          )
        )
      </CardContent>
    </Card>
  );
}

```



```

        <DialogContent>
          <DialogHeader>
            <DialogTitle>Reject Document</DialogTitle>
            <DialogDescription>
              Provide a reason for rejecting this
{document.type}.
            </DialogDescription>
          </DialogHeader>
          <Textarea>
            placeholder="Enter rejection reason..."
            value={rejectionReason}
            onChange={(e) =>
setRejectionReason(e.target.value)}
            rows={4}
          </>
          <DialogFooter>
            <Button variant="outline" onClick={() =>
setDialogOpen(false)}>
              Cancel
            </Button>
            <Button>
              variant="destructive"
              onClick={handleReject}
              disabled={!rejectionReason.trim()}
            >
              Confirm Rejection
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
    </div>
  </div>
  )}
</CardContent>
</Card>
);
}

```

5. SOSEventCard

```

// components/sos/sos-event-card.tsx
"use client";

import { useEffect, useState } from "react";
import { Card, CardContent } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Avatar, AvatarFallback, AvatarImage } from "@components/
ui/avatar";
import { MapPin, Clock, ExternalLink } from "lucide-react";
import Link from "next/link";

interface SOSUser {

```

```

    id: string;
    name: string;
    photo_url?: string;
  }

interface SOSEventProps {
  event: {
    id: string;
    user: SOSUser;
    guard: SOSUser;
    booking_id: string;
    location: { lat: number; lng: number };
    triggered_at: string;
  };
  onResolve: (id: string) => void;
}

function formatElapsed(seconds: number): string {
  const hrs = Math.floor(seconds / 3600);
  const mins = Math.floor((seconds % 3600) / 60);
  const secs = seconds % 60;
  if (hrs > 0) return `${hrs}h ${mins}m ${secs}s`;
  if (mins > 0) return `${mins}m ${secs}s`;
  return `${secs}s`;
}

export function SOSEventCard({ event, onResolve }: SOSEventProps)
{
  const [elapsed, setElapsed] = useState(0);

  useEffect(() => {
    const triggeredTime = new Date(event.triggered_at).getTime();
    const update = () => setElapsed(Math.floor((Date.now() -
triggeredTime) / 1000));
    update();
    const interval = setInterval(update, 1000);
    return () => clearInterval(interval);
  }, [event.triggered_at]);

  const mapsUrl = `https://maps.googleapis.com/maps/api/staticmap?
center=${event.location.lat},${event.location.lng}
&zoom=15&size=300x150&markers=color:red%7C${event.location.lat},${
event.location.lng}&key=$
{process.env.NEXT_PUBLIC_GOOGLE_MAPS_KEY}`;

  return (
    <Card className="border-l-4 border-l-red-500">
      <CardContent className="p-4 space-y-3">
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-3">
            <div className="flex -space-x-2">
              <Avatar className="h-8 w-8 border-2 border-white">
                <AvatarImage src={event.user.photo_url} />
                <AvatarFallback>{event.user.name[0]}</

```

```

AvatarFallback>
    </Avatar>
    <Avatar className="h-8 w-8 border-2 border-white">
        <AvatarImage src={event.guard.photo_url} />
        <AvatarFallback>{event.guard.name[0]}</
AvatarFallback>
    </Avatar>
</div>
<div>
    <p className="text-sm font-medium">{event.user.name}</p>
</div>
<p>
    <p className="text-xs text-muted-foreground">Guard:
{event.guard.name}</p>
</div>
</div>
<div className="flex items-center gap-1 text-red-600
font-mono text-sm font-bold">
    <Clock className="h-4 w-4" />
    {formatElapsed(elapsed)}
</div>
</div>

<img
    src={mapsUrl}
    alt="SOS Location"
    className="w-full h-[100px] rounded-md object-cover"
/>

<div className="flex items-center justify-between">
    <Link
        href={` /admin/bookings/${event.booking_id}`}
        className="text-xs text-blue-600 hover:underline flex
items-center gap-1"
    >
        <ExternalLink className="h-3 w-3" />
        Booking #{event.booking_id.slice(0, 8)}
    </Link>
    <div className="flex items-center gap-1 text-xs text-
muted-foreground">
        <MapPin className="h-3 w-3" />
        {event.location.lat.toFixed(4)},
{event.location.lng.toFixed(4)}
    </div>
</div>

<Button
    className="w-full"
    variant="destructive"
    onClick={() => onResolve(event.id)}
>
    Resolve SOS
</Button>
</CardContent>
</Card>

```

```
);  
}
```

6. BookingTimeline

```
// components/bookings/booking-timeline.tsx  
"use client";  
  
import { formatDistanceToNow } from "date-fns";  
import { cn } from "@lib/utils";  
  
interface TimelineEvent {  
  status: string;  
  timestamp: string;  
  note?: string;  
}  
  
interface BookingTimelineProps {  
  events: TimelineEvent[];  
}  
  
const statusDotColor: Record<string, string> = {  
  pending: "bg-yellow-500",  
  confirmed: "bg-blue-500",  
  guard_assigned: "bg-indigo-500",  
  guard_en_route: "bg-purple-500",  
  in_progress: "bg-green-500",  
  completed: "bg-green-700",  
  cancelled: "bg-red-500",  
  refunded: "bg-orange-500",  
};  
  
function formatStatus(status: string): string {  
  return status.replace(/_/g, " ").replace(/\b\w/g, (c) =>  
    c.toUpperCase());  
}  
  
export function BookingTimeline({ events }: BookingTimelineProps)  
{  
  return (  
    <div className="space-y-0">  
      {events.map((event, index) => {  
        const isLast = index === events.length - 1;  
        const dotColor = statusDotColor[event.status] ?? "bg-gray-400";  
  
        return (  
          <div key={index} className="flex gap-3">  
            <div className="flex flex-col items-center">  
              <div className={cn("h-3 w-3 rounded-full mt-1.5 shrink-0", dotColor)} />  
            <div>  
              {formatStatus(event.status)}  
              {event.timestamp}</div>  
            <div>  
              {event.note}</div>  
          </div>  
        )  
      })}  
    </div>  
  )  
}
```

```

        {!isLast && <div className="w-px flex-1 bg-border" /
    >}
        </div>
        <div className={cn("pb-6", isLast && "pb-0")}>
            <p className="text-sm font-
medium">{formatStatus(event.status)}</p>
            <p className="text-xs text-muted-foreground">
                {formatDistanceToNow(new Date(event.timestamp),
{ addSuffix: true })}
            </p>
            {event.note && (
                <p className="mt-1 text-xs text-muted-foreground
italic">{event.note}</p>
            )}
        </div>
    </div>
    );
    })}
</div>
);
}

```

7. PageHeader

```

// components/layout/page-header.tsx
import { ReactNode } from "react";

interface PageHeaderProps {
    title: string;
    description?: string;
    action?: ReactNode;
}

export function PageHeader({ title, description, action }:
PageHeaderProps) {
    return (
        <div className="flex items-center justify-between">
            <div>
                <h1 className="text-2xl font-bold tracking-tight">{title}
</h1>
                {description && (
                    <p className="text-sm text-muted-foreground
mt-1">{description}</p>
                )}
            </div>
            {action && <div>{action}</div>}
        </div>
    );
}

```

8. DateRangePicker

```
// components/ui/date-range-picker.tsx
"use client";

import { useState } from "react";
import { format, subDays, startOfMonth, startOfDay, endOfDay }
from "date-fns";
import { CalendarIcon } from "lucide-react";
import { Button } from "@components/ui/button";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { Calendar } from "@components/ui/calendar";
import { cn } from "@lib/utils";
import { DateRange } from "react-day-picker";

interface DateRangePickerProps {
  value: { from: Date; to: Date };
  onChange: (range: { from: Date; to: Date }) => void;
}

export function DateRangePicker({ value, onChange }:
DateRangePickerProps) {
  const [open, setOpen] = useState(false);

  const presets = [
    {
      label: "Today",
      getValue: () => ({ from: startOfDay(new Date()), to:
endOfDay(new Date()) }),
    },
    {
      label: "Last 7 days",
      getValue: () => ({ from: subDays(new Date(), 7), to: new
Date() }),
    },
    {
      label: "Last 30 days",
      getValue: () => ({ from: subDays(new Date(), 30), to: new
Date() }),
    },
    {
      label: "This Month",
      getValue: () => ({ from: startOfMonth(new Date()), to: new
Date() }),
    },
  ];

  const handleSelect = (range: DateRange | undefined) => {
    if (range?.from && range?.to) {
      onChange({ from: range.from, to: range.to });
    }
  };
};
```

```

const applyPreset = (preset: (typeof presets)[number]) => {
  onChange(preset.getValue());
  setOpen(false);
};

return (
  <Popover open={open} onOpenChange={setOpen}>
    <PopoverTrigger asChild>
      <Button variant="outline" className={cn("justify-start
text-left font-normal w-[260px]")}>
        <CalendarIcon className="mr-2 h-4 w-4" />
        {format(value.from, "MMM d, yyyy")} - {format(value.to,
"MMM d, yyyy")}
      </Button>
    </PopoverTrigger>
    <PopoverContent className="w-auto p-0" align="start">
      <div className="flex">
        <div className="border-r p-3 space-y-1">
          {presets.map((preset) => (
            <Button
              key={preset.label}
              variant="ghost"
              size="sm"
              className="w-full justify-start text-xs"
              onClick={() => applyPreset(preset)}
            >
              {preset.label}
            </Button>
          ))}
        </div>
        <div className="p-3">
          <Calendar
            mode="range"
            selected={{ from: value.from, to: value.to }}
            onSelect={handleSelect}
            numberOfMonths={2}
            defaultMonth={value.from}
          />
        </div>
      </div>
    </PopoverContent>
  </Popover>
);
}

```

9. ConfirmDialog

```

// components/ui/confirm-dialog.tsx
"use client";

```

```

import { ReactNode } from "react";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from "@components/ui/alert-dialog";
import { cn } from "@lib/utils";
import { Loader2 } from "lucide-react";

interface ConfirmDialogProps {
  title: string;
  description: string;
  confirmLabel?: string;
  variant?: "default" | "destructive";
  isLoading?: boolean;
  onConfirm: () => void;
  trigger: ReactNode;
}

export function ConfirmDialog({
  title,
  description,
  confirmLabel = "Confirm",
  variant = "default",
  isLoading = false,
  onConfirm,
  trigger,
}: ConfirmDialogProps) {
  return (
    <AlertDialog>
      <AlertDialogTrigger asChild>{trigger}</AlertDialogTrigger>
      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>{title}</AlertDialogTitle>
          <AlertDialogDescription>{description}</
AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel disabled={isLoading}>Cancel</
AlertDialogCancel>
          <AlertDialogAction
            onClick={onConfirm}
            disabled={isLoading}
            className={cn(
              variant === "destructive" &&
                "bg-destructive text-destructive-foreground
hover:bg-destructive/90"
            )}>

```



```

        >
        {isLoading && <Loader2 className="mr-2 h-4 w-4
animate-spin" />}
        {confirmLabel}
        </AlertDialogAction>
        </AlertDialogFooter>
        </AlertDialogContent>
        </AlertDialog>
    );
}

```

10. ExportButton

```

// components/ui/export-button.tsx
"use client";

import { useState } from "react";
import { Button } from "@components/ui/button";
import { Download, Loader2 } from "lucide-react";
import { api } from "@services/api";

interface ExportButtonProps {
  endpoint: string;
  filename: string;
  filters?: Record<string, any>;
}

export function ExportButton({ endpoint, filename, filters }:
ExportButtonProps) {
  const [isLoading, setIsLoading] = useState(false);

  const handleExport = async () => {
    setIsLoading(true);
    try {
      const response = await api.get(endpoint, {
        params: filters,
        responseType: "blob",
        headers: { Accept: "text/csv" },
      });

      const blob = new Blob([response.data], { type: "text/
csv" });
      const url = window.URL.createObjectURL(blob);
      const link = document.createElement("a");
      link.href = url;
      link.download = `${filename}_${new
Date().toISOString().split("T")[0]}.csv`;
      document.body.appendChild(link);
      link.click();
      document.body.removeChild(link);
      window.URL.revokeObjectURL(url);
    }
  };
}

```

```

    } catch (error) {
      console.error("Export failed:", error);
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <Button variant="outline" size="sm" onClick={handleExport}
disabled={isLoading}>
      {isLoading ? (
        <Loader2 className="mr-2 h-4 w-4 animate-spin" />
      ) : (
        <Download className="mr-2 h-4 w-4" />
      )}
      Export CSV
    </Button>
  );
}

```